

Enhanced Cross Residual Graph Neural Networks for Graph Classification

Your Name

Department/Institution

University/Organization

City, Country

email@example.com

March 12, 2026

Abstract

Deep graph classification benefits from residual information flow, but the exact value of cross-branch interaction depends strongly on how node-level and graph-level signals are coupled in the implementation. This paper studies Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a modular family of six architectures built from three message-passing operators (GCNConv, GATConv, and TransformerConv) and two reusable residual mechanisms: intra-branch residual accumulation and cross-branch residual exchange. Rather than describing the framework abstractly, we align the method section with the executed PyTorch Geometric implementation and derive the experimental conclusions directly from the raw record exports under `records/`. We verify 24 full v3 record files (720 configurations) and incorporate a newer v4 rerun that replaces the older MUTAG+GCNConv slice with 60 updated measurements. Across the merged benchmark, ResBlockGnn achieves the best overall mean accuracy (0.775), CrossGraphBlockGnn attains the strongest AIDS result (0.815), and residual variants consistently improve depth tolerance over the plain BlockGNN baseline. The updated MUTAG+GCNConv rerun further reveals a more nuanced picture: graph-level residual passing is strongest at shallow depth on this slice, whereas ResBlockGnn remains the most stable architecture overall. These results suggest that residual design matters more reliably than simply increasing branch count, and that cross-branch fusion is most useful when matched to dataset scale and operator bias.

1 Introduction

Graph classification is a fundamental task in graph representation learning, with applications in molecular property prediction, drug discovery, social network analysis, and computer vision. Unlike node-level prediction, graph classification requires the model to summarize an entire graph into a single discriminative representation. This setting amplifies several well-known difficulties of graph neural networks (GNNs): graphs vary in size and topology, local neighborhoods can be noisy, long-range dependencies are hard to preserve, and deep message passing often suffers from over-smoothing and unstable optimization.

Modern GNNs address these challenges through a diverse set of message-passing operators. Graph Convolutional Networks (GCN) [1] emphasize smooth neighborhood aggregation, Graph Attention Networks (GAT) [2] learn adaptive neighbor weights, and graph Transformers [3] broaden the receptive field through attention. Despite their differences, these operators are usually studied as alternative backbones inside a largely sequential architecture. As depth increases, repeated aggregation can still blur node representations and weaken the useful diversity of intermediate features [4–6].

Several research directions have tried to alleviate this problem. Residual connections [7, 8], normalization [9, 10], and edge dropout [11] stabilize deep training. Multi-scale aggregation methods such as Jumping Knowledge [12] and dense connectivity [13] preserve information from multiple depths. Pooling methods such as DiffPool [14] and SAGPool [15] add hierarchical structure. However, most of these designs still operate within a single branch and do not explicitly compare different ways of reusing information across branches and graph-level summaries inside one implementation.

This paper studies Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a modular framework that compares several residual and cross-residual mechanisms under the same PyTorch Geometric codebase. Instead of treating the method as a single monolithic model, we view it as a family of six related graph classifiers built from the same operator factory and training pipeline. The family spans plain stacking, intra-branch residual reuse, node-level cross residual, sequential graph-level conditioning, repeated graph-level residual refinement, and graph-level cross residual between block pairs. This implementation-oriented view makes it possible to ask a more practical question: when does extra cross-branch interaction actually help beyond a simpler residual baseline?

The contributions of this work are summarized as follows:

- We formalize ECR-GNN as a unified implementation family rather than a single abstract architecture, and describe the method directly from the executed code path.
- We distinguish two reusable information-flow mechanisms: node-level cross residual between parallel branches and graph-level residual propagation between sequential blocks.
- We perform a record-verified empirical study on four TUDataset benchmarks, covering six architectures, three operators, five depths, and two widths. The results show that residual reuse is consistently beneficial, while cross-branch interaction helps mainly in dataset-specific regimes.
- We provide a manuscript-ready analysis that integrates raw record validation, a merged rerun for the updated MUTAG + GCNConv slice, regenerated figures and tables, and implementation-aligned pseudocode.

2 Related Work

We review related work from five perspectives: graph neural networks for graph classification, message passing architectures and operators, deep GNN training challenges, multi-operator

and hybrid architectures, and cross-layer aggregation mechanisms. Our work builds upon these foundations by introducing a unified framework that integrates multiple graph operators through cross-branch residual connections.

2.1 Graph Neural Networks for Graph Classification

Graph-level prediction requires aggregating node-level features into fixed-size graph representations. Early graph neural networks [16] laid the foundation for learning on graph-structured data through recursive message passing. The Message Passing Neural Networks (MPNN) framework [17] unified various GNN architectures under a common formalism, identifying message and vertex update functions as core components.

For graph classification, readout functions play a crucial role in converting node embeddings to graph-level representations. Simple approaches apply global pooling operations such as mean or sum pooling [18]. However, these methods may not adequately capture hierarchical structural information. To address this, differentiable pooling methods have been proposed: DiffPool [14] learns node cluster assignments to coarsen graphs hierarchically, SAGPool [15] employs self-attention to identify salient nodes, and LocalPool [19] enables sparse hierarchical classification. Recent surveys provide comprehensive overviews of graph pooling techniques [20, 21]. While these approaches achieve strong performance, they primarily focus on intra-branch architectural designs and do not explicitly consider how multiple operators with different inductive biases can be integrated and coordinated through cross-branch information exchange.

2.2 Message Passing Architectures and Operators

Various message passing operators have been developed, each imposing different inductive biases on neighborhood aggregation. Graph Convolutional Networks (GCN) [1] approximate spectral convolutions through normalized adjacency matrices, performing low-pass filtering that smooths node features. GraphSAGE [22] samples and aggregates features from local neighborhoods using mean, LSTM, or pooling aggregators, enabling inductive learning on unseen graphs. Graph Isomorphism Networks (GIN) [6] use injective aggregation functions to achieve maximum discriminative power, matching the expressive power of the Weisfeiler-Lehman test.

Attention mechanisms have been widely incorporated to weight neighbor contributions adaptively. Graph Attention Networks (GAT) [2] compute attention coefficients for each neighbor, allowing nodes to focus on relevant information. Personalized Graph Neural Networks (PGNN) [23] further extend attention mechanisms for personalized aggregation. Recent work generalizes Transformer architectures to graphs [3] and incorporates edge directionality [24, 25] for improved modeling of directed and heterophilic graphs.

Other operator variants include ARMA filters [26] for flexible frequency response, Mix-Hop [27] for mixing multi-hop neighborhoods, and anti-symmetric convolutions [28] for stable deep architectures. While these operators are typically studied and deployed individually within homogeneous architectures, a critical gap remains: existing frameworks treat operators as alternative choices rather than complementary components that can be systematically combined and coordinated to leverage their diverse inductive biases within a unified model.

2.3 Deep GNN Training and Representation Degradation

Training deep GNNs faces fundamental challenges due to representation degradation. The *over-smoothing* phenomenon causes node representations to converge toward indistinguishable values as depth increases [4, 5, 29]. Theoretical analysis [29] provides non-asymptotic bounds on over-smoothing, revealing fundamental limitations in expressive power for deep architectures. Additionally, *over-squashing* [30, 31] occurs when long-range dependencies cannot be effectively propagated due to graph bottlenecks, further constraining model performance.

To enable deep training, various techniques have been proposed. Residual connections [7], originally developed for computer vision, have been adapted to GNNs through residual gated graph convolutions [8]. Normalization methods such as PairNorm [9] and GraphNorm [10] alleviate over-smoothing by normalizing node features, while learnable normalization [32] adapts to different graph distributions. Regularization techniques like DropEdge [11] randomly drop edges during training to prevent overfitting and improve generalization.

Advanced architectures attempt to train extremely deep networks: techniques for training 1000-layer GNNs [33] and simplified deep GCN approaches [34] demonstrate that depth can improve performance with proper design. However, these methods primarily operate within single-branch architectures and do not explicitly model how information can be exchanged across multiple operator-specific branches to preserve multi-scale features and mitigate degradation at depth.

2.4 Multi-Operator and Hybrid Architectures

Combining multiple aggregation schemes or architectural components has shown promise in enhancing representational capacity. Multi-hop attention [35] enables information flow across longer distances, while ARMA filters [26] provide flexible frequency responses beyond polynomial filters. Hybrid architectures incorporate edge directionality [24, 25] to capture directed relationships, and anti-symmetric convolutions [28] improve stability in deep networks.

Surveys on attention-based GNNs [36] and graph embeddings [37] provide comprehensive taxonomies of architectural variants. DenseGNN [13] connects all layers densely, similar to DenseNet in computer vision, to maintain information flow. Hierarchical pooling methods [14, 38] create multi-scale representations through graph coarsening. However, most existing multi-operator approaches treat different components as modular plug-ins that are combined sequentially or in parallel, without explicit mechanisms for structured information exchange between them during training. Our work addresses this gap by introducing cross-branch residual connections that enable operators to continuously exchange and refine information throughout the forward pass.

2.5 Cross-Layer and Multi-Scale Aggregation

Cross-layer connections aim to capture multi-scale information by aggregating features from different depths. Jumping Knowledge (JK) networks [12] adaptively select which layer’s representation to use for each node, enabling flexible neighborhood range selection. Graph U-Net [39] adopts encoder-decoder architecture with skip connections between corresponding layers.

Propagation-based methods improve information flow across multiple hops: APPNP [40] combines personalized PageRank with neural predictions, while PairNorm [9] and DropEdge [11] mitigate over-smoothing through normalization and regularization. Recent work on over-smoothing [5, 41] provides comprehensive analysis and solutions.

Despite these advances, most cross-layer methods operate within a single homogeneous branch using one type of operator. A fundamental limitation is that they do not consider how multiple branches with different operators can leverage complementary inductive biases through structured cross-branch communication. Our framework introduces cross-branch residual connections that enable node-level and graph-level information exchange between operator-specific branches, facilitating synergistic collaboration where, for example, the smoothing properties of GCN can stabilize the attention weights of GAT, while the selectivity of GAT can refine the low-pass features of GCN. This cross-branch coordination, combined with multi-scale aggregation within each branch, provides a more comprehensive approach to integrating diverse architectural biases for improved graph classification.

3 Task Definition

We formally define the graph classification problem and establish notation for our cross-residual framework that integrates multiple graph operators. Unlike traditional approaches that focus on a single operator (e.g., GCN or GAT), our framework treats graph convolution operators as modular components that can be systematically combined through cross-branch residual connections.

3.1 Graph Representation and Notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, where d_{in} is the input feature dimension. The node features for the entire graph are collected in a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, where the i -th row corresponds to $\mathbf{x}_i$.

The graph structure is represented using an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ if there exists an edge $(v_i, v_j) \in \mathcal{E}$, and $A_{ij} = 0$ otherwise. For undirected graphs, the adjacency matrix is symmetric ($\mathbf{A} = \mathbf{A}^\top$). Optionally, edge features can be represented as $\mathbf{e}_{uv} \in \mathbb{R}^{d_e}$ for each edge $(u, v) \in \mathcal{E}$, or collected in a tensor $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$.

3.2 Graph Classification Problem

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$ consisting of N graphs, where each graph $\mathcal{G}_i$ is associated with a label $y_i \in \mathcal{Y}$, our objective is to learn a mapping function $f_\theta : \mathcal{G} \rightarrow \hat{y}$ that predicts the label $\hat{y} \in \mathcal{Y}$ for an unseen graph. The label space $\mathcal{Y}$ depends on the task type:

- **Binary classification:** $\mathcal{Y} = \{0, 1\}$ (e.g., molecular toxicity prediction)
- **Multi-class classification:** $\mathcal{Y} = \{1, 2, \dots, C\}$ (e.g., graph categorization into C classes)
- **Multi-label classification:** $\mathcal{Y} \subseteq \{1, 2, \dots, C\}$ (e.g., a graph may belong to multiple categories simultaneously)

The function f_θ is parameterized by neural network weights θ and operates by first learning node-level representations through message passing, then aggregating these into a graph-level representation, and finally mapping this representation to the output label space.

3.3 Message Passing with Multiple Operators

Our framework supports multiple message passing operators $\{\Phi_1, \Phi_2, \dots, \Phi_K\}$, where each operator Φ_k imposes a distinct inductive bias on neighborhood aggregation. Common operators include:

- **GCN** [1]: Normalized adjacency-based aggregation with low-pass filtering
- **GAT** [2]: Attention-weighted aggregation with learnable importance coefficients
- **GraphSAGE** [22]: Sampling-based aggregation with mean/LSTM/pooling operators
- **GIN** [6]: Injective aggregation for maximum discriminative power
- **Transformer** [3]: Self-attention over nodes for long-range dependencies

For a single operator Φ_k , node representations are updated over L layers. At layer ℓ , the representation $\mathbf{h}_i^{(\ell,k)}$ of node v_i in branch k is computed as:

$$\mathbf{h}_i^{(\ell+1,k)} = \sigma \left(\mathbf{W}^{(\ell,k)} \cdot \text{AGGREGATE}_k \left(\left\{ \mathbf{h}_j^{(\ell,k)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell,k)} \right) + \mathbf{b}^{(\ell,k)} \right) \quad (1)$$

where AGGREGATE_k is the aggregation function specific to operator Φ_k (e.g., mean pooling for GCN, attention weights for GAT), $\mathbf{W}^{(\ell,k)}$ are learnable weights, $\mathbf{b}^{(\ell,k)}$ is a bias term, and σ is a non-linear activation function (e.g., ReLU). Our cross-residual framework enhances this standard formulation by introducing cross-branch connections that enable information exchange between different operators Φ_k and $\Phi_{k'}$ at each layer, as detailed in Section 4.

3.4 Graph-Level Representation Learning

After L layers of message passing across K branches, we obtain node representations $\{\mathbf{h}_i^{(L,k)} : i = 1, \dots, n\}$ for each branch k . A readout function R_k aggregates node-level representations into a branch-specific graph-level embedding $\mathbf{h}_{\mathcal{G}}^{(k)} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{h}_{\mathcal{G}}^{(k)} = R_k \left(\left\{ \mathbf{h}_i^{(L,k)} : i = 1, \dots, n \right\} \right) \quad (2)$$

Common readout functions include global mean pooling ($\frac{1}{n} \sum_{i=1}^n \mathbf{h}_i^{(L,k)}$), max pooling ($\max_{i=1, \dots, n} \mathbf{h}_i^{(L,k)}$), and sum pooling ($\sum_{i=1}^n \mathbf{h}_i^{(L,k)}$). In our framework, the final graph-level representation $\mathbf{h}_{\mathcal{G}}$ combines embeddings from all K branches through cross-residual connections, enabling the model to leverage complementary inductive biases from different operators.

3.5 Learning Objective

The graph-level representation is passed through a classifier (typically a linear layer followed by softmax) to produce the predicted label distribution:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}}\mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}) \quad (3)$$

where $\hat{\mathbf{y}} \in \mathbb{R}^C$ (for multi-class classification) is the predicted probability distribution over C classes, and $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{C \times d_{\text{out}}}$, $\mathbf{b}_{\text{clf}} \in \mathbb{R}^C$ are classifier parameters.

The model is trained end-to-end by minimizing the cross-entropy loss function:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (4)$$

where $y_{i,c} \in \{0, 1\}$ is the ground-truth label indicator (one-hot encoding) for graph $\mathcal{G}_i$ and class c , and $\hat{y}_{i,c}$ is the predicted probability for class c . For binary classification, this reduces to the binary cross-entropy loss. The parameters θ include all weights in the message passing layers, readout functions, cross-residual connections, and the classifier, which are optimized using gradient-based methods (e.g., Adam) with backpropagation.

4 Proposed Model

ECR-GNN is implemented as a compact family of reusable graph classification modules. Instead of presenting an idealized design that differs from the executed code, this section follows the actual PyTorch Geometric implementation in `geomatric/graph_classify_v2.py`. The implementation exposes one operator factory and six model variants, all of which share the same input projection, stacked message-passing layers, global mean pooling, and linear graph classifier.

4.1 Operator Factory and Shared Backbone

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The operator factory instantiates one of three message-passing layers,

$$\Phi \in \{\text{GCNConv}, \text{GATConv}, \text{TransformerConv}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d , every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{MeanPool}(\mathbf{H}^{(L)}). \quad (6)$$

The final graph prediction is produced by

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{clf}} \text{Dropout}(\mathbf{g}) + \mathbf{b}_{\text{clf}}. \quad (7)$$

All six architectures differ only in how they update $\mathbf{H}^{(\ell)}$ and how they reuse graph-level embeddings across blocks.

4.2 Single-Branch and Node-Level Residual Blocks

BlockGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \text{Dropout}\left(\text{ReLU}\left(\Phi_\ell\left(\mathbf{H}^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (8)$$

This model serves as the reference point for judging whether residual information flow is beneficial.

ResBlockGnn. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \text{Dropout}\left(\text{ReLU}\left(\Phi_\ell\left(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\right)\right)\right). \quad (9)$$

This design matches the code path where `x_pre` is cached and added back before each layer call. The mechanism is lightweight and preserves historical features without creating extra branches.

CrossBlockGnn. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \text{Dropout}\left(\text{ReLU}\left(\Phi_\ell^{(1)}\left(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (10)$$

$$\mathbf{H}_2^{(\ell+1)} = \text{Dropout}\left(\text{ReLU}\left(\Phi_\ell^{(2)}\left(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (11)$$

where $\tilde{\mathbf{H}}_k^{(\ell)}$ is the cached hidden state from the opposite branch before its current update. The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (12)$$

Compared with the single-branch residual block, CrossBlockGnn explicitly exchanges information between operator-aligned pathways at every depth.

4.3 Graph-Level Residual Propagation

The code also defines three higher-level architectures that compose several block models sequentially and pass graph embeddings across them.

GraphBlockGnn. Two plain BlockGNN modules are executed in sequence. The first block produces a graph embedding $\mathbf{g}^{(1)}$, which is added to node features and to the pooled graph representation inside the second block:

$$\mathbf{g}^{(2)} = B_2\left(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(1)}\right). \quad (13)$$

This introduces graph-level conditioning without intra-layer residual connections.

ResGraphBlockGnn. This variant chains three graph-conditioned BlockGNN modules:

$$\mathbf{g}^{(t+1)} = B_{t+1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\right), \quad t \in \{1, 2\}. \quad (14)$$

Inside each block, the graph embedding is broadcast back to node states before the next graph convolution. The design can be viewed as repeated graph-level residual refinement.

CrossGraphBlockGnn. The strongest graph-level interaction appears in the cross-graph variant. Four BlockGNN modules are organized as two sequential pairs. Within each stage, the two block outputs are swapped and reused as the next stage’s graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}), B_{2t}(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}) \right), \quad (15)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \left(\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)} \right). \quad (16)$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (17)$$

This is the implementation-level meaning of graph-level cross residual in our experiments: branches do not exchange node tensors directly, but they do swap pooled graph summaries across stages.

4.4 Unified Training Objective

All model variants use the same graph classification objective:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c}, \quad (18)$$

optimized with Adam. Dropout is applied after every message-passing update and once more before the classifier. The experiments in Section 6 therefore isolate architectural differences rather than changes in loss, optimizer, or data protocol.

4.5 Algorithmic View

The following pseudocode summarizes the implementation-level forward logic shared across the experiments.

Algorithm 1. Forward propagation of ECR-GNN

- 1: **Input:** graph batch $(\mathbf{X}, \mathbf{A}, \mathbf{b})$, operator type Φ , depth L , architecture type
- 2: Instantiate block operators through the factory `GETBLOCKMODEL( $\Phi$ )`
- 3: **if** architecture is `BLOCKGNN` **then**
- 4: Apply one input projection and L stacked operator layers
- 5: **else if** architecture is `RESBLOCKGNN` **then**
- 6: Cache the previous hidden state and add it back before each layer update
- 7: **else if** architecture is `CROSSBLOCKGNN` **then**
- 8: Build two branches; at each depth inject the cached hidden state from the opposite branch
- 9: **else if** architecture is `GRAPHBLOCKGNN` or `RESGRAPHBLOCKGNN` **then**
- 10: Run multiple block models sequentially and reuse the previous graph embedding as graph-level context
- 11: **else if** architecture is `CROSSGRAPHBLOCKGNN` **then**
- 12: **for** each block pair **do**
- 13: Compute two graph embeddings in parallel
- 14: Swap the two graph embeddings before the next pair

15: **end for**
16: **end if**
17: Pool node embeddings with global mean pooling
18: Apply dropout and the linear classifier to obtain $\hat{\mathbf{y}}$
19: **Return:** logits $\hat{\mathbf{y}}$ and pooled graph embedding $\mathbf{g}$

4.6 Design Implications

This implementation leads to three practical hypotheses that are later verified by experiments. First, residual reuse should matter more consistently than simply adding more branches, because every successful variant still relies on repeated feature reuse. Second, node-level cross residual and graph-level residual passing are not interchangeable: the former exchanges intermediate node representations, whereas the latter only re-injects pooled graph summaries. Third, the usefulness of cross-branch fusion depends on dataset scale and operator bias; it is therefore necessary to analyze accuracy, stability, and efficiency jointly rather than reporting only the single best score.

5 Datasets

To comprehensively evaluate the effectiveness of ECR-GNN, we conduct experiments on four benchmark datasets from the TU Dortmund University (TUDataset) collection [42]. These datasets vary in size, domain, and graph characteristics, providing a robust testbed for evaluating cross-residual mechanisms and multi-operator architectures.

5.1 Dataset Overview

We use the following datasets in our experiments:

MUTAG MUTAG is a standard benchmark dataset for molecular property prediction, consisting of 188 chemical compounds representing nitroaromatic molecules. Each graph corresponds to a molecule, where nodes represent atoms and edges represent chemical bonds. The task is binary classification: predicting whether a molecule has mutagenic effects on the Gram-negative bacterium *Salmonella typhimurium*. This dataset is widely used in graph classification literature due to its small size and well-defined task, making it suitable for controlled ablation studies.

DD The DD (Dreiding-Dataset) dataset contains 1,178 chemical compounds representing small organic molecules. Each graph encodes the molecular structure of a compound, with nodes representing atoms and edges representing chemical bonds. The task is to predict whether a compound is active against drug-resistant HIV strains. Compared to MUTAG, DD presents a larger-scale challenge with more graphs and complex molecular structures, testing the model’s ability to handle increased dataset size and structural diversity.

Table 1: Summary of graph classification datasets used in experiments

Dataset	# Graphs	# Classes	Feature Dim.	Domain	Task Type
MUTAG	188	2	Dataset-specific TU features	Bio	Binary
DD	1,178	2	Dataset-specific TU features	Bio	Binary
MSRC_9	231	9	Dataset-specific TU features	Vision	Multi-class
AIDS	2,000	2	Dataset-specific TU features	Bio	Binary

MSRC_9 MSRC_9 (Microsoft Research Cambridge 9) is a dataset derived from image segmentation tasks, containing 231 graphs representing image regions. Each graph corresponds to a segmented image from the MSRC dataset, where nodes represent image regions and edges indicate spatial adjacency between regions. The task is multi-class classification: assigning each graph to one of 9 scene categories (e.g., building, tree, cow, airplane). This dataset introduces non-biological domain data, testing the generalizability of our framework beyond molecular graphs.

AIDS AIDS is a bioinformatics dataset containing 2,000 chemical compounds screened for activity against HIV. Each graph represents a molecule, with nodes as atoms and edges as chemical bonds. The task is binary classification: predicting whether a compound is active against HIV. This is the largest dataset in our experiments, challenging the model’s scalability and ability to learn from diverse molecular structures.

5.2 Dataset Statistics

Table 1 summarizes the key statistics of the datasets used in our experiments. Statistics are computed from the TUDataset loader in PyTorch Geometric.

5.3 Data Loading and Configuration

All datasets are loaded using the TUDataset loader from PyTorch Geometric [43]. The data loading process follows the implementation in `geomatric/graph_classify_v2.py`:

Dataset Source Datasets are loaded from the TUDataset collection with the root path configured as `/data/ai_data` on Linux or `../data` on Windows. Each dataset is specified by name (e.g., ‘‘MUTAG”, ‘‘DD”, ‘‘MSRC_9”, ‘‘AIDS”) and automatically downloaded to the configured path if not present.

Graph Representation Each graph is represented as a PyTorch Geometric `Data` object containing:

- **x**: Node feature matrix of shape $(n \times d_{\text{in}})$, where n is the number of nodes and d_{in} is the input feature dimension specific to the dataset
- **edge_index**: Graph connectivity in COO format, shape $(2, |\mathcal{E}|)$, where $|\mathcal{E}|$ is the number of edges

- **y**: Graph-level label (scalar for binary/multi-class classification)
- **batch**: Batch assignment vector (for mini-batch processing)

Node Features Node features are provided by the TUDataset loader in their original form. No additional feature engineering or normalization is applied during data loading. The feature dimension d_{in} is dataset-specific and automatically inferred by the loader. For molecular datasets (MUTAG, DD, AIDS), features typically encode atom types and chemical properties; for MSRC_9, features encode visual descriptors of image regions.

Edge Attributes and Self-Loops **Edge attributes** are not used in our implementation. All graph convolution operators (GCN, GAT, Transformer) operate on unweighted graphs. **Self-loops** are added automatically by specific operators when required (e.g., GCN adds self-loops via $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$), but this is handled internally by the operator implementation rather than as a preprocessing step.

5.4 Data Splitting Protocol

We use 5-fold cross-validation for all datasets to ensure robust evaluation and enable fair comparison with baseline methods.

5-Fold Cross-Validation The dataset is randomly shuffled and split into 5 equal-sized folds using the implementation in `load_data(start_index)` (line 167-188 in `graph_classify_v2.py`). For each fold $k \in \{0, 1, 2, 3, 4\}$:

1. Fold k is held out as the test set
2. The remaining 4 folds are concatenated to form the training set
3. The model is trained on the training set and evaluated on the test set
4. This process is repeated for all 5 folds, and the mean accuracy and standard deviation across folds are reported

The split is implemented as:

$$\begin{aligned} \text{train_dataset} &= \mathcal{D}[0 : k \cdot \text{split_len}] \cup \mathcal{D}[(k + 1) \cdot \text{split_len} : |\mathcal{D}|] \\ \text{test_dataset} &= \mathcal{D}[k \cdot \text{split_len} : (k + 1) \cdot \text{split_len}] \end{aligned} \tag{19}$$

where $\mathcal{D}$ is the full dataset, $\text{split_len} = |\mathcal{D}|/5$, and k is the fold index.

Reproducibility To ensure reproducibility, random seeds are set before training:

- PyTorch random seed: 1024
- NumPy random seed: 1024
- Python random seed: 1024

- CUDA random seed: 1024 (if GPU is available)

The dataset shuffling is deterministic given the fixed random seed, ensuring that the 5-fold splits are consistent across different experimental runs.

No Official Splits Used We do not use the predefined splits provided by TUDataset. Instead, we generate our own 5-fold splits to maintain consistency with the experimental setup in prior work and to enable controlled comparison across different model architectures. All results report mean accuracy and standard deviation computed from the 5 folds.

5.5 Data Preprocessing and Augmentation

Minimal Preprocessing Our framework applies minimal preprocessing to the raw datasets:

- **Dataset shuffling:** The dataset is shuffled once using the fixed random seed before creating 5-fold splits
- **Batching:** Graphs are grouped into batches of size 32 using PyTorch Geometric’s `DataLoader`
- **No feature normalization:** Node features are used as-is without scaling or centering
- **No graph augmentation:** No edge dropout, node dropping, or subgraph sampling is applied

Batching Strategy Training and test sets are loaded using separate `DataLoader` instances:

- **Training loader:** `DataLoader(train_dataset, batch_size=32, shuffle=True)` - shuffles batches each epoch
- **Test loader:** `DataLoader(test_dataset, batch_size=32, shuffle=False)` - maintains deterministic order for evaluation

The batch size of 32 is used across all datasets and models, as specified in the implementation.

Regularization While not data augmentation per se, we apply **dropout** regularization during training:

- Dropout probability: 0.6 (default, configurable via `--drop` argument)
- Applied after each message passing layer
- Applied before the final classifier

Dropout acts as a form of implicit data augmentation by randomly dropping node features during training, which helps prevent overfitting despite the small dataset sizes.

5.6 Evaluation Metrics

Following standard practice in graph classification literature, we evaluate model performance using:

Accuracy Accuracy is the primary evaluation metric, computed as the proportion of correctly classified graphs:

$$\text{Accuracy} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i=1}^{|\mathcal{D}_{\text{test}}|} \mathbb{I}[\hat{y}_i = y_i] \quad (20)$$

where $\mathcal{D}_{\text{test}}$ is the test set, $\hat{y}_i$ is the predicted label, and y_i is the ground-truth label.

Aggregation Across Folds For each dataset, we report:

- **Mean accuracy:** $\mu = \frac{1}{5} \sum_{k=0}^4 \text{acc}_k$, where acc_k is the accuracy on fold k
- **Standard deviation:** $\sigma = \sqrt{\frac{1}{5} \sum_{k=0}^4 (\text{acc}_k - \mu)^2}$

These statistics are computed across the 5 folds, providing a robust measure of model performance and stability. Results are reported in the format “mean $\pm$ standard deviation” (e.g., “0.85 $\pm$ 0.03”).

6 Experiments

This section is organized in the same result-first spirit as the reference paper: we first report the main benchmark results, then analyze the updated rerun slice in detail, and finally discuss stability and efficiency. All numbers are derived directly from the raw record exports under `records/`; we do not manually rewrite experimental values.

6.1 Experimental Settings

6.1.1 Data Sources and Verification

The experiment archive contains two summary spreadsheets, `v3result.xlsx` and `v4result.xlsx`, together with their corresponding raw record files. We verified that the 24 v3 record files contain exactly 720 non-empty lines and that the 2 v4 record files contain exactly 60 non-empty lines, matching the spreadsheet row counts one-to-one. The v4 archive is not an additional experiment set; it is a rerun of the MUTAG + GCNConv slice. Therefore, our final analysis uses a merged benchmark in which the 60-row v4 rerun replaces the older MUTAG + GCNConv rows from v3, resulting in 720 final configurations.

6.1.2 Configuration Space

We evaluate six architectures:

1. **BlockGNN**: plain stacked message passing
2. **ResBlockGnn**: single-branch residual reuse
3. **CrossBlockGnn**: two-branch node-level cross residual
4. **GraphBlockGnn**: sequential graph-level conditioning
5. **ResGraphBlockGnn**: repeated graph-level residual refinement

Table 2: Best 5-fold graph classification accuracy (mean $\pm$ std) for each architecture on each dataset after replacing the older MUTAG+GCNConv slice with the v4 rerun.

Model	MUTAG	DD	MSRC_9	AIDS
BlockGNN	0.822 $\pm$ 0.0404	0.618 $\pm$ 0.0211	0.982 $\pm$ 0.0170	0.810 $\pm$ 0.0213
ResBlockGnn	0.832 $\pm$ 0.0315	0.611 $\pm$ 0.0316	0.982 $\pm$ 0.0265	0.812 $\pm$ 0.0068
CrossBlockGnn	0.811 $\pm$ 0.0705	0.617 $\pm$ 0.0222	0.977 $\pm$ 0.0144	0.812 $\pm$ 0.0218
GraphBlockGnn	0.827 $\pm$ 0.0653	0.609 $\pm$ 0.0277	0.977 $\pm$ 0.0144	0.812 $\pm$ 0.0173
ResGraphBlockGnn	0.816 $\pm$ 0.0465	0.619 $\pm$ 0.0384	0.973 $\pm$ 0.0223	0.812 $\pm$ 0.0083
CrossGraphBlockGnn	0.778 $\pm$ 0.0315	0.614 $\pm$ 0.0275	0.982 $\pm$ 0.0170	0.815 $\pm$ 0.0172

Table 3: Best configuration of each architecture on the updated MUTAG + GCNConv rerun slice (derived from v4 records).

Model	Dim	Depth	Acc.	Std.	Time (s)
BlockGNN	32	1	0.800	0.0930	577.4
ResBlockGnn	32	4	0.811	0.0419	1018.8
CrossBlockGnn	64	2	0.811	0.0705	1842.4
GraphBlockGnn	32	2	0.827	0.0653	1228.7
ResGraphBlockGnn	32	2	0.805	0.0733	1229.5
CrossGraphBlockGnn	32	1	0.778	0.0315	1634.5

6. CrossGraphBlockGnn: graph-level cross residual between sequential block pairs

Each architecture is instantiated with three operators (GCNConv, GATConv, TransformerConv), five depths ($h \in \{1, 2, 3, 4, 5\}$), and two hidden dimensions (32 and 64). The full grid therefore contains $6 \times 3 \times 5 \times 2 \times 4 = 720$ evaluated settings across the four datasets MUTAG, DD, MSRC_9, and AIDS.

6.1.3 Training Protocol

All runs follow the implementation in `geomatric/graph_classify_v2.py`: 5-fold cross-validation, Adam optimizer, learning rate 0.01, weight decay 10^{-2} , dropout 0.6, and up to 1500 epochs with early stopping once the training loss falls below 0.001. We report mean accuracy, fold standard deviation, and total execution time recorded by the experiment scripts.

6.2 Main Results

Table 2 summarizes the best configuration of each architecture on each dataset. Figure 1 shows the average accuracy over all 720 merged configurations, while Figure 2 visualizes the best accuracy reached by each architecture on each dataset.

Three observations stand out.

Residual reuse is the most reliable gain. Averaged over the full configuration space, ResBlockGnn achieves the best mean accuracy of 0.775, ahead of CrossBlockGnn (0.766), GraphBlockGnn (0.748), BlockGNN (0.740), and CrossGraphBlockGnn (0.734). This ranking indicates that explicit feature reuse is more consistently beneficial than simply increasing architectural complexity.

Table 4: Efficiency comparison on MUTAG with GCNConv, dim=32, and depth=2. Relative overhead is measured against BlockGNN.

Model	Time (s)	Relative Overhead	Accuracy
BlockGNN	709.8	+0.0%	0.708
ResBlockGnn	738.9	+4.1%	0.795
CrossBlockGnn	1158.0	+63.1%	0.724
GraphBlockGnn	1228.7	+73.1%	0.827
ResGraphBlockGnn	1229.5	+73.2%	0.805
CrossGraphBlockGnn	2187.2	+208.1%	0.622

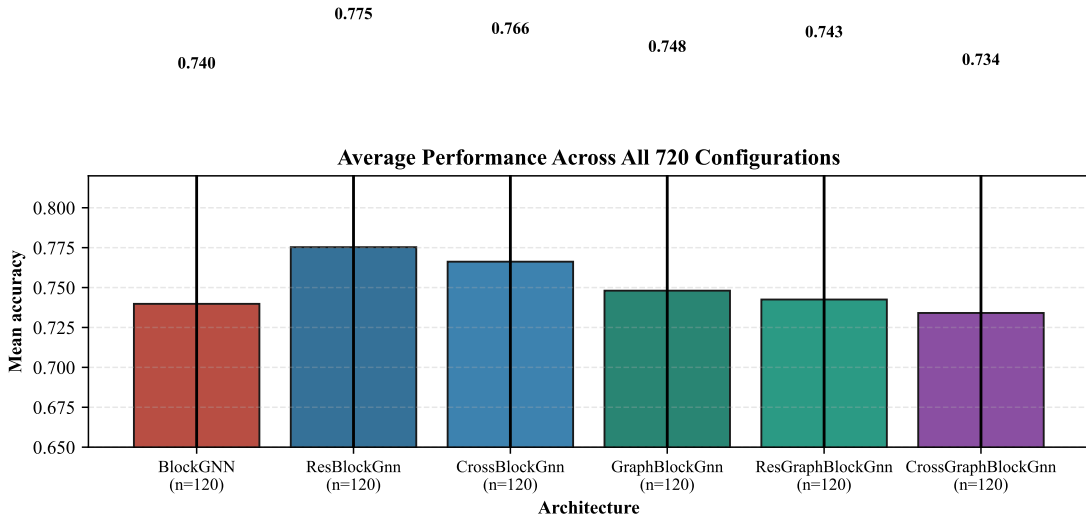


Figure 1: Average accuracy across all 720 merged configurations. ResBlockGnn achieves the highest global mean accuracy (0.775), followed by CrossBlockGnn (0.766).

The best architecture depends on the dataset. ResBlockGnn reaches the strongest MUTAG result (0.832 with TransformerConv, dim=64, depth=2), ResGraphBlockGnn gives the best DD score (0.619 with TransformerConv, dim=64, depth=2), and CrossGraphBlockGnn obtains the best AIDS score (0.815 with GATConv, dim=32, depth=2). On MSRC_9, three architectures reach the same top accuracy of 0.982, showing that this dataset is already close to saturation for several model families.

Dataset difficulty dominates absolute performance. The mean accuracy over all settings is 0.898 on MSRC_9, 0.799 on AIDS, 0.719 on MUTAG, and only 0.587 on DD. The DD improvements over the best BlockGNN baseline are therefore extremely small (0.14% relative), whereas MUTAG and AIDS still benefit from architectural choices.

6.3 Updated MUTAG + GCNConv Rerun Analysis

The v4 rerun is especially useful because it refreshes the full MUTAG + GCNConv slice with both widths and all depths. Table 3 lists the best rerun configuration of each architecture, and Figure 3 shows the full depth curves for both widths.

The rerun reveals a more nuanced pattern than a single headline score would suggest.

Graph-level conditioning can be strongest at shallow depth. On the dim=32 branch,

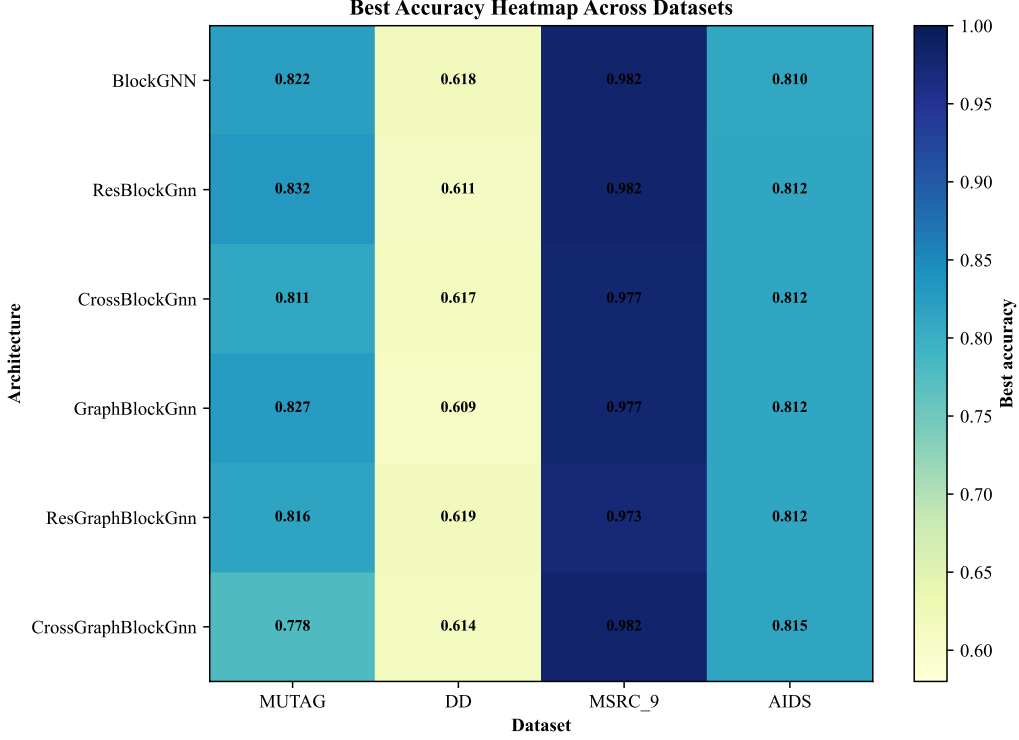


Figure 2: Best accuracy heatmap across datasets. MSRC_9 is consistently easy, DD remains the hardest benchmark, and AIDS shows only a narrow spread between the strongest architectures.

GraphBlockGnn peaks at 0.827 with depth 2, outperforming all other GCN-based variants in that slice. ResGraphBlockGnn is also strong at depth 2 (0.805), which means graph-level residual reuse is genuinely effective when the model remains shallow.

ResBlockGnn is the most depth-tolerant architecture in the rerun. On the same dim=32 slice, BlockGnn drops from 0.800 at depth 1 to 0.600 at depth 5, a 25.0% relative degradation. In contrast, ResBlockGnn reaches its peak at depth 4 (0.811) and still retains 0.773 at depth 5, showing that intra-branch residual accumulation is the most reliable safeguard against performance collapse.

Node-level cross residual is competitive but less stable. On the dim=64 slice, CrossBlockGnn reaches 0.811 at depth 2, matching the best GCN rerun score of ResBlockGnn. However, its fold standard deviation at that point is 0.070, clearly larger than the 0.030 observed for ResBlockGnn in the same setting. Cross-branch exchange can therefore raise the ceiling, but it does not automatically improve robustness.

6.4 Stability and Efficiency

To separate average accuracy from robustness, Figure 4 plots the five-fold distribution on the shared slice MUTAG + GCNConv with dim=64 and depth=2. Table 4 reports execution time and relative overhead on the commonly referenced MUTAG + GCNConv setting with dim=32 and depth=2.

Two complementary conclusions follow.

ResBlockGnn is the most stable architecture overall. Across all 720 merged con-

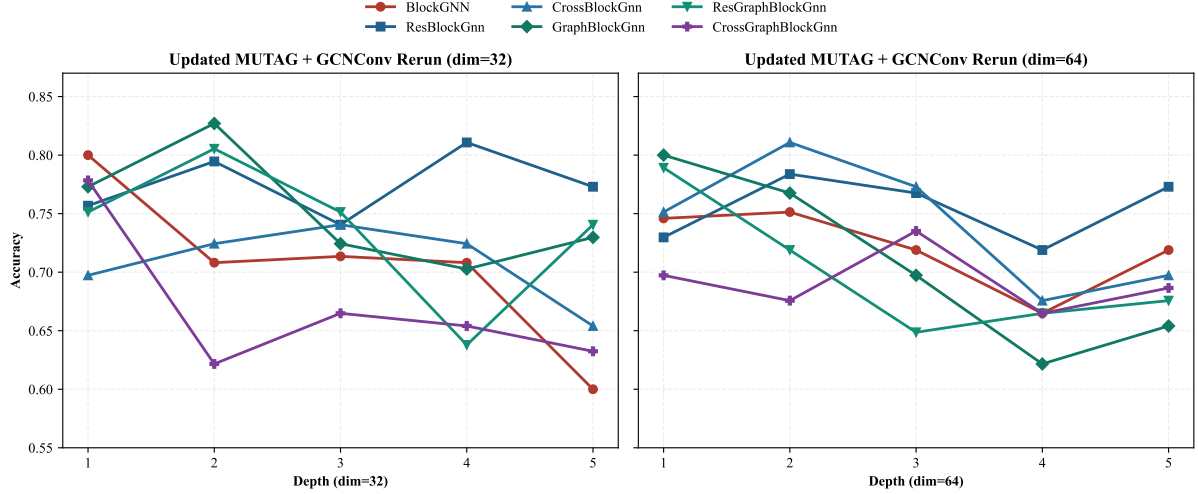


Figure 3: Updated depth sensitivity on the rerun MUTAG + GCNConv slice. Dim=32 and dim=64 show different winners, which confirms that architecture comparisons should not rely on a single depth-width setting.

figurations, it has the lowest mean fold standard deviation (0.0312), narrowly ahead of Cross-BlockGnn (0.0316), and clearly better than BlockGNN (0.0516). The box plot shows the same behavior on the shared rerun slice: ResBlockGnn keeps a tight fold range while remaining highly competitive in mean accuracy.

Efficiency sharply penalizes the more complex graph-level variants. On MUTAG + GCNConv with dim=32 and depth=2, ResBlockGnn improves accuracy from 0.708 to 0.795 with only 4.1% extra time. GraphBlockGnn reaches the strongest accuracy on that slice (0.827) but needs 73.1% more time, while CrossGraphBlockGnn is both slower (+208.1%) and weaker (0.622). The full-grid timing statistics tell the same story: average execution time rises from 9136 seconds for BlockGNN to 9572 seconds for ResBlockGnn, but jumps to 31266 seconds for CrossGraphBlockGnn.

6.5 Summary of Findings

The merged record analysis supports four final claims. First, residual reuse is a stronger general-purpose design choice than branch proliferation. Second, graph-level residual propagation is useful, but mainly in specific shallow or dataset-dependent regimes rather than as a universal winner. Third, node-level cross residual can reach strong peak accuracy, yet its gains come with higher variance and cost. Fourth, once accuracy, stability, and execution time are considered together, ResBlockGnn offers the best overall trade-off in this repository, while Cross-GraphBlockGnn should be treated as a specialized model for datasets such as AIDS where its graph-level interaction is actually rewarded.

7 Conclusion

This paper revisited ECR-GNN from an implementation-grounded perspective and evaluated it with a verified experiment archive rather than a hand-curated summary. By checking the

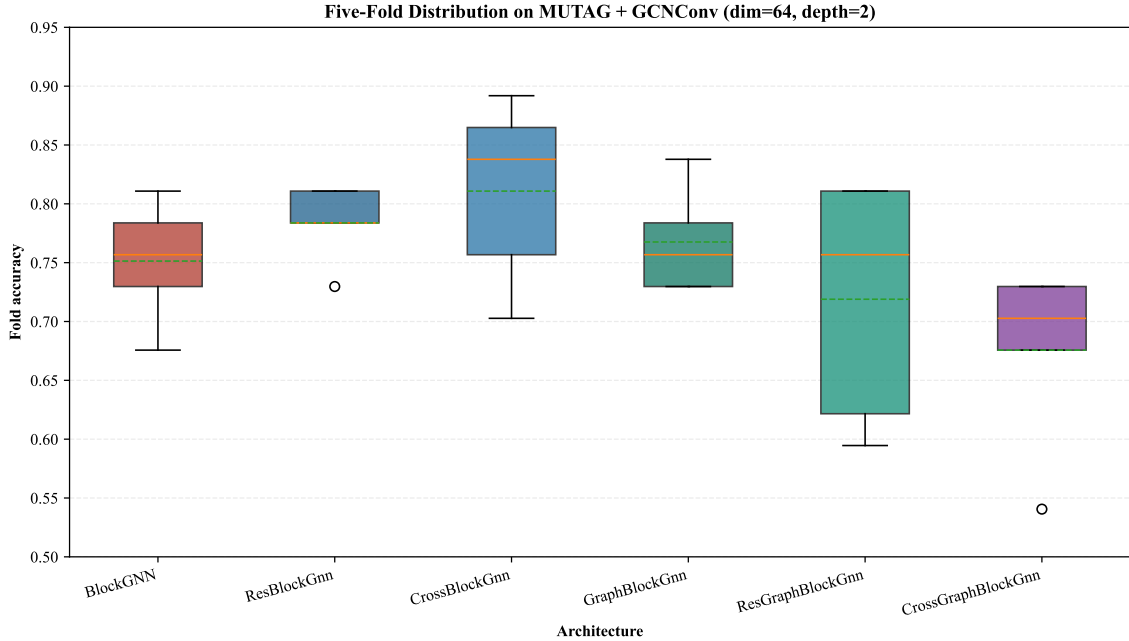


Figure 4: Fold-wise accuracy distribution on MUTAG + GCNConv (dim=64, depth=2). ResBlockGnn has the tightest spread, whereas CrossBlockGnn attains a high mean with noticeably larger variance.

raw record exports and merging the newer v4 rerun into the earlier v3 benchmark, we obtained a consistent 720-configuration result set for comparing six residual and cross-residual graph classification architectures.

The empirical picture is clear. ResBlockGnn is the strongest all-around design: it achieves the best global mean accuracy (0.775), the lowest average fold variance, and only a small execution-time increase over BlockGNN. CrossBlockGnn can reach similarly strong peak scores on selected slices, but it is less stable. Graph-level interaction is still useful, especially for AIDS and for shallow MUTAG + GCNConv settings, yet it is not universally superior and becomes expensive when implemented through multiple sequential block models. In particular, CrossGraphBlockGnn delivers the best AIDS score (0.815) but also incurs the largest average runtime.

The updated MUTAG + GCNConv rerun adds an important nuance: the locally best architecture within one operator-dataset slice is not always the globally best architecture across the full benchmark. GraphBlockGnn is strongest on the shallow dim=32 rerun slice, whereas ResBlockGnn remains the most depth-tolerant and stable model overall. This reinforces the main practical takeaway of the paper: residual design should be judged jointly by accuracy, robustness, and cost, not by a single best-case number.

Future work can move in three directions. First, the graph-level variants should be re-designed to reduce their substantial runtime overhead. Second, heterogeneous operator combinations deserve a direct study, since the current experiments still keep the operator type homogeneous within each model instance. Third, larger datasets and stronger statistical testing are needed to determine when cross-branch interaction offers a meaningful advantage beyond the simpler residual baseline.

References

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [3] V. P. Dwivedi and X. Bresson, “A generalization of transformer networks to graphs,” *arXiv preprint arXiv:2012.09699*, 2020.
- [4] G. Li, E. Muller, A. Thabet, and B. Ghanem, “Deeper insights into graph convolutional networks for semi-supervised learning,” *arXiv preprint arXiv:1809.02584*, 2018.
- [5] Y. Chen, X. Wei, P. Xu, X. Wang, P. Luo, Z. Li, and J. Tang, “Revisiting over-smoothing in deep gcns,” *arXiv preprint arXiv:2003.13663*, 2020.
- [6] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *International Conference on Learning Representations (ICLR)*, 2019.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [8] X. Bresson and T. Laurent, “Residual gated graph convnets,” *arXiv preprint arXiv:1711.07553*, 2017.
- [9] L. Zhao and L. Akoglu, “Pairnorm: Tackling over-smoothing in gns,” *arXiv preprint arXiv:2009.10698*, 2020.
- [10] T. Cai, J. Luo, S. Wang, K. Zhang, Y. Tian, L. Yang, Z. Li, and K. Weinberger, “Graph-norm: A principled approach to accelerating graph neural network training,” in *International Conference on Machine Learning*, 2021, pp. 1277–1287.
- [11] Y. Rong, W. Huang, T. Xu, and J. Huang, “Dropedge: Towards deep graph convolutional networks on node classification,” *arXiv preprint arXiv:1907.10903*, 2019.
- [12] K. Xu, C. Li, Y. Tian, X. Du, J. Leskovec, and S. Jegelka, “Representation learning on graphs with jumping knowledge networks,” in *International Conference on Machine Learning*, 2018, pp. 5449–5458.
- [13] H. Du, Y. Zhang, P. Zhang, B. Gu, X. Wei, J. Guo, A. Li, Z. Wang, T. Ma *et al.*, “Densegcn: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules,” *npj Computational Materials*, vol. 10, no. 1, p. 304, 2024.
- [14] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, “Hierarchical graph representation learning with differentiable pooling,” in *Advances in neural information processing systems*, 2018, pp. 4800–4810.

- [15] J. Lee, I. Lee, and J. Kang, “Self-attention graph pooling,” *arXiv preprint arXiv:1904.08082*, 2019.
- [16] M. Gori, G. Monfardini, and F. Scarselli, “A new model for learning in graph domains,” *Proceedings of IJCNN*, 2005.
- [17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *International Conference on Machine Learning*, 2017, pp. 1263–1272.
- [18] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, “Convolutional networks on graphs for learning molecular fingerprints,” in *Advances in neural information processing systems*, 2015, pp. 2224–2232.
- [19] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Liò, “Towards sparse hierarchical graph classifiers,” in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2018, pp. 165–176.
- [20] C. Liu, X. Wang, Y. Li, Y. Wang, and J. Gao, “Graph pooling for graph neural networks,” pp. 4254–4261, 2023.
- [21] Z. Li, H. Chen, Z. Zeng, J. He, X. Mao, N. Lv, and L. Chen, “Graph pooling for graph-level representation learning: a survey,” *Artificial Intelligence Review*, vol. 57, no. 8, pp. 1–51, 2024.
- [22] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in neural information processing systems*, 2017, pp. 1024–1034.
- [23] M. Zhang, Y. Liu, Q. Chen, and S. Pan, “Personalized graph neural networks with attention mechanism,” 2019.
- [24] D. Beaini, S. Passaro, E. Létouzey, J. Domke, S. Erdos, P. Lió, P. Veličković, and M. M. Bronstein, “Directional graph networks,” *arXiv preprint arXiv:2010.02863*, 2020.
- [25] E. Rossi, F. Frasca, F. M. Bianchi, N. Alletto, and P. Lio, “Edge directionality improves learning on heterophilic graphs,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [26] F. M. Bianchi, F. Grattarola, and C. Alippi, “Graph neural networks with convolutional arma filters,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [27] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, G. Steeg, and A. Galstyan, “Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing,” in *International Conference on Machine Learning*, 2019, pp. 33–42.
- [28] A. Gravina, N. Alletto, A. Rossi, F. M. Bianchi, and P. Lio, “Anti-symmetric dgn: A stable architecture for deep graph networks,” *arXiv preprint arXiv:2210.09789*, 2022.

- [29] S. Li, M. Zhang, M. Jin, H. Zhang, R. Ying, Z. Li, and J. Yan, “A non-asymptotic analysis of oversmoothing in graph neural networks,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [30] J. Topping, F. Di Giovanni, B. Chamberlain, M. Bronstein, D. Vendrig, and P. Lio, “Understanding over-squashing and bottlenecks on graphs via curvature,” *arXiv preprint arXiv:2111.14522*, 2021.
- [31] U. Alon and E. Yahav, “On the bottleneck of graph neural networks,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [32] Y. Li, X. Wang, Z. Wang, and J. Li, “Learning graph normalization for graph neural networks,” *Neurocomputing*, vol. 486, pp. 140–151, 2022.
- [33] G. Li, E. Müller, A. Thabet, and B. Ghanem, “Training graph neural networks with 1000 layers,” in *International Conference on Machine Learning*, 2021, pp. 6403–6413.
- [34] K. Oono and T. Suzuki, “Simple and deep graph convolutional networks,” *arXiv preprint arXiv:2006.13604*, 2020.
- [35] G. Wang, I. Balazevic, M. Znis, M. Gitta, Y. Kwon, M. Seeger, X. Li, L. De Raedt, and M. Welling, “Multi-hop attention graph neural networks,” in *International Joint Conference on Artificial Intelligence*, 2021, pp. 2112–2120.
- [36] X. Sun, R. Wang, L. Cui, X. Kong, and S. Li, “Attention-based graph neural networks: a survey,” *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 493–13 539, 2023.
- [37] J. Cai, X. Zheng, C. Luo, C. Yang, and E. Santus, “Understanding graph embedding methods and their applications,” *arXiv preprint arXiv:2012.08019*, 2020.
- [38] H. Van Pham, U. The, T. Do, and A. Nguyen, “Hierarchical pooling in graph neural networks to enhance graph classification,” *Applied Sciences*, vol. 11, no. 17, p. 8164, 2021.
- [39] H. Gao and S. Ji, “Graph u-net,” *arXiv preprint arXiv:1905.05178*, 2019.
- [40] J. Klicpera, A. Bojchevski, and S. Günnemann, “Combining label propagation and simple models out-performs graph neural networks,” *arXiv preprint arXiv:1810.05997*, 2018.
- [41] L. Zhao, Z. Leng, Y. Chen, and L. Akoglu, “Tackling over-smoothing for general graph learning,” *arXiv preprint arXiv:2008.09864*, 2020.
- [42] N. Keriven, G. Peyré, and S. Vaiter, “A unified benchmark for the diagnosis and treatment of graph neural networks,” *arXiv preprint arXiv:2012.01450*, 2020.
- [43] M. Fey and J. E. Lenssen, “Fast graph representation learning with pytorch geometric,” *arXiv preprint arXiv:1903.02428*, 2019.